



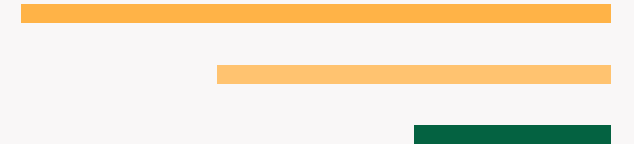
LifeData Analytics Hub

Workflows

Prepared by: Lifewood PH Team

Date: August 14, 2026

CONFIDENTIAL – FOR INTERNAL USE ONLY



Contents

1.0 Development Workflow	3
2.0 Build & Deploy Pipeline	4
3.0 Adding a New Dashboard	5
4.0 Adding a New Affiliation	6
5.0 Code Quality & Linting	7
6.0 Troubleshooting Workflows	8

How to use this document

Audience: Developers and contributors who build, extend, or deploy this platform. Assumes working knowledge of the listed technologies but no prior familiarity with this specific codebase.

Conventions: monospace text = literal commands or code you type; < angle brackets > = replace with your value; callout boxes highlight warnings or prerequisites.

Development Workflow

The LifeData project follows a trunk-based development workflow with feature branches and pull requests.

1. Create a branch from main: `git checkout -b feat/my-feature`; use descriptive names like `fix/login-error` or `feat/byu-ethiopia`.
2. Make small, focused commits in present tense, e.g. 'Add BYU Ethiopia dashboard config'.
3. Push the branch, open a PR against main, and ensure `npm run lint` && `npm run build` pass first.

GIT WORKFLOW

```
git checkout -b feat/new-dashboard  
git commit -m "Add new dashboard"  
git push origin feat/new-dashboard
```

No test command currently exists — a test framework (Vitest) should be added before writing tests.

Build & Deploy Pipeline

LifeData uses Vercel for continuous deployment – every push to main triggers an automatic deployment.

- 1. Run npm run lint and npm run build – fix ESLint errors and TypeScript failures.
- 2. Verify the build output in the dist/ directory.
- 3. Commit and push to the main branch.
- 4. Monitor the Vercel deployment and check the production URL.

Step	Command	Duration	Tool
Lint	npm run lint	~10s	ESLint 9
Type check	npm run build (includes tsc)	~30s	TypeScript 6
Build	npm run build	~20s	Vite 7
Deploy	git push	~60s	Vercel CI
Verify	Manual	~30s	Browser

The build pipeline produces optimized static assets in dist/ – bundling, code splitting, and fingerprinting by Vite. No Dockerfile: Vercel serves the static build directly.

3.0

Adding a New Dashboard

Adding a new country or project dashboard requires zero component code changes – the GenericTableDashboard auto-renders from configuration.

1. Create the Supabase table from your CSV or XLSX data.
2. Enable Row-Level Security and add a SELECT policy for anon or authenticated users.
3. Add a TableDashboardConfig entry in src/config/tableDashboards.ts.
4. Push to main – deployment is automatic.
5. Verify the new dashboard appears in the sidebar.

EXAMPLE CONFIG

```
{
  tabId: 'byu-ethiopia',
  tableId: 'BYUEthiopia',
  label: 'Ethiopia',
  sidebarFolder: 'byu',
  flagCode: 'ET',
  title: 'BYU Ethiopia',
  subtitle: 'Ethiopia project participants',
  columns: [...],
}
```

4.0

Adding a New Affiliation

Affiliations map participant organizations and are defined in `crowdsourceAffiliations.ts`. Adding a new affiliation is a config-only change.

1. Open `src/config/crowdsourceAffiliations.ts`.
2. Add the new affiliation name to the appropriate list (`PH_AFFILIATION_NAMES` or `INTL_AFFILIATION_NAMES`).
3. Optionally add a flag mapping in `affilFlags.ts`.
4. Verify the affiliation appears in filter dropdowns, then push to main for deployment.

AFFILIATION CONFIG

```
export const PH_AFFILIATION_NAMES = new Set([
  'University of Santo Tomas',
  'De La Salle University',
  'New Affiliation Here', // Add new entry
]);
```

Note

Affiliations use case-insensitive matching. The system auto-merges variations like 'Student ID' and 'Student Number' through built-in data normalization.

Code Quality & Linting

ESLint 9 with the flat config (eslint.config.js) enforces code quality. Run checks before every commit.

Rule Set	Purpose
eslint:recommended	Standard JS error prevention
react-hooks + react-refresh + no-unused-vars	Hooks rules, Vite HMR compatibility, unused variable detection

QUALITY COMMANDS

```
# Run lint
npm run lint
npm run build # includes TypeScript check
```

1. TypeScript strict mode (strict: true in tsconfig.json).
2. React functional components with hooks – no class components.
3. Tailwind CSS for styling, no inline styles; no semicolons, single quotes, 2-space indentation.

Troubleshooting Workflows

Issue	Cause	Fix
npm run build fails with TS errors	Type mismatch or missing imports	Run <code>tsc --noEmit</code> for the full error list, then fix types.
npm run lint fails with react-hooks errors	Hook dependency array issue	Add missing deps to <code>useEffect</code> dependency arrays.
Vercel deploy fails	Build step failed	Check Vercel dashboard build logs; reproduce locally.
New dashboard not appearing after deploy	Config syntax error or missing Supabase table	Verify the <code>tableDashboards.ts</code> entry; check the table exists.
Git push rejected	Branch behind main or permission denied	<code>git pull --rebase origin main</code> , resolve conflicts.
Build succeeds but blank page on prod	Missing or incorrect env variables on Vercel	Check <code>VITE_SUPABASE_URL</code> and <code>VITE_SUPABASE_ANON_KEY</code> .